Metasploit: steps to check that it works

Updated: 2 November 2026 (Cairo)

Run these steps in order. Each step says what a correct result looks like. Stop and fix any step that does not match before going on.

> Do not run Metasploit against any public website. No public site allows it in this project.

Step 0. Open the project terminal

Open the project folder in VS Code (the folder that contains `backend`, `docs` and `scripts`). Open **Terminal → New Terminal**. Every command below runs from the **project root**, except where it says **backend**.

Step 1. Activate the project's Python environment

If `backend/.venv` does not exist yet, create it first (see the beginner guide). Do not delete an existing `.venv`.

Mac (zsh):

```
cd backend
source .venv/bin/activate
pip install -r requirements.txt -r requirements-dev.txt
cd ..
```

Windows (PowerShell):

```
Set-Location backend
.\.venv\Scripts\Activate.ps1
pip install -r requirements.txt -r requirements-dev.txt
Set-Location ..
```

Expected: pip finishes without errors.

Step 2. Run the framework unit tests (no database, no Metasploit)

These tests use a built-in test double, `FakeMsfrpcd`. The double speaks the same MessagePack protocol but does not run Metasploit. They confirm the adapter and scanner logic.

Mac (zsh):

```
cd backend
./.venv/bin/python -m pytest --noconftest -q -k "not persist" tests/unit/test_pentest_tools.py tests/
unit/test_pentest_framework.py tests/unit/test_pentest_safety_regressions.py tests/unit/
test_sqlmap_integration.py tests/unit/test_public_tool_results.py
cd ..
```

Windows (PowerShell):

```
Set-Location backend
.\.venv\Scripts\python.exe -m pytest --noconftest -q -k "not persist" tests/unit/test_pentest_tools.py
tests/unit/test_pentest_framework.py tests/unit/test_pentest_safety_regressions.py tests/unit/
test_sqlmap_integration.py tests/unit/test_public_tool_results.py
Set-Location ..
```

Expected: `228 passed, 2 deselected`. The `persist` tests are skipped because they need a database. This result was recorded on Linux. On Mac or Windows, the count should match; if it does not, note the first failing test name.

> Do not run the full test suite against your real database. The general test fixture drops tables. Use a separate disposable database if you run it at all.

Step 3. Check the simulated Metasploit path (beginner lab)

This runs a local lab and a **simulated** Metasploit response. It does not start Metasploit.

Mac (zsh):

```
./backend/.venv/bin/python scripts/live_tests/beginner_tools_lab.py metasploit
```

Windows (PowerShell):

```
.\backend\.venv\Scripts\python.exe scripts\live_tests\beginner_tools_lab.py metasploit
```

Expected: a line that says `SIMULATED`, and a `PASS` line for parsing the fake response. A `PASS` here does **not** prove that real Metasploit works. Only Step 5 does that.

Step 4. Install and start the real daemon (optional but required for Step 5)

Follow **metasploit how to use.pdf**, sections 2 and 3. Start `msfrpcd` on `127.0.0.1` port `55553`, and keep its window open.

Check that it is listening.

Mac (zsh):

```
lsof -nP -iTCP:55553 -sTCP:LISTEN
```

Windows (PowerShell):

```
Get-NetTCPConnection -LocalPort 55553 -State Listen
```

Expected: one listening entry on `127.0.0.1` (or `::1`) port 55553. If nothing appears, the daemon did not start. Check the password and the install path before continuing.

Set `MSF_ENABLED=true` and the other `MSF_RPC_*` values in `backend/.env` (see the how-to), then restart the backend if it is running.

Step 5. Run the real Metasploit port scan through Cerberus

Save the example from **metasploit how to use.pdf**, section 5, as `msf_check.py` inside the **backend** folder.

Mac (zsh):

```
cd backend
./.venv/bin/python msf_check.py
cd ..
```

Windows (PowerShell):

```
Set-Location backend
.\.venv\Scripts\python.exe msf_check.py
Set-Location ..
```

Expected:

- If **nothing** listens on ports 22, 80 or 443 on your computer, the result is **zero open ports**. That is correct.
- If a service listens on one of those ports, you should see a finding such as `Metasploit auxiliary/scanner/portscan/tcp: 1 open ports on 127.0.0.1`.
- If you see `Metasploit integration disabled`, the `.env` setting is not active. Go back to Step 4.
- If you see a timeout, wait 30 seconds and run it again. A cold daemon can take about 20 seconds to start its first console. Do not increase the timeout below 60 seconds.

To see a real open port, first start a temporary listener in another terminal (for example `python -m http.server 8080`) and change the example to scan port 8080. Then delete the listener when you are done.

Step 6. Stop and clean up

1. Stop `msfrpcd` in its window with `Ctrl+C`.
2. Delete `backend/msf_check.py` if you no longer need it.
3. Keep `backend/.env` private. Do not paste the password into chat, a screenshot, or a commit.

Step 7. What to report if something fails

Note the step number, the exact command, and the last 20 lines of output. Do not include the password or your `.env` contents.

If you see	Likely cause
`command not found` for `msfrpcd` or `msfconsole`	Metasploit is not installed, or it is not on PATH
Connection refused on port 55553	The daemon is not running, or it uses a different port
Authentication failed	`MSF_RPC_PASSWORD` in `.env` does not match the `-P` value
Out of scope or refused module	Expected for anything except `auxiliary/scanner/portscan/tcp`
Timeout on first console	Cold start. Retry once; keep `MSF_RPC_TIMEOUT=60`

Result summary

Check	Meaning if it passes
Step 2	The adapter and scanner code behave as tested (test double, not the real program)

Check	Meaning if it passes
Step 3	The lab and parser work (simulated response)
Step 5	Cerberus talked to a **real** Metasploit daemon on your computer

Public Metasploit testing: **not permitted by any public site used in this project.**